

MixBytes()

[SECURITY AUDIT REPORT]

Shift USDT0 Bridge & Strategies

PREPARED FOR SHIFT DEFI

REPORT DATE

July 31, 2026

Table of Contents

1. Introduction	4
1.1 Disclaimer	4
1.2 Executive Summary	4
1.3 Project Overview	8
1.4 Security Audit Methodology	10
1.5 Risk Classification	12
1.6 Summary of Findings	13
2. Findings Report	16
2.1 Critical	16
C-1 Source-chain compose injection on the destination adapter	16
2.2 High	18
H-1 IzCompose rejects the LayerZero Endpoint and misreads the OFT envelope	18
H-2 Outbound bridge sends USDT0 to the peer container instead of the destination adapter	20
2.3 Medium	21
M-1 Inherited retryBridge creates unnecessary replay and payload-substitution surface	21
M-2 lastStoredGaugeBalance snapshot not refreshed after treasury transfers in _harvest	23
M-3 Reward compounding swaps the full reward balance at a zero slippage floor	25
M-4 Merkle claimer can redeposit parked Morpho principal outside the recorded state	26
M-5 Harvest reinvest leg reverts at the vault deposit cap, blocking deposits and withdrawals	27
2.4 Low	28
L-1 CRV emissions are never claimed and non-pool-coin reward tokens are left idle	28
L-2 Harvest fee underflows on virtual_price decrease, blocking deposits and withdrawals	29
L-3 payload.refundRecipient and payload.gasLimit are not validated	30
L-4 Harvest fee misses virtual-price growth accrued outside the gauge state	31
L-5 Initialization does not verify cross-field consistency	32
L-6 Dormant pool coins are excluded from _curveGaugeNav and _curveLpNav	33
L-7 Math.mulDiv reverts when ICurveStableSwapNG.totalSupply() == 0	34
L-8 Uninitialised lastStored* charges fee on principal	35
L-9 Compose payload encodes instruction.amount instead of the OFT-delivered amount	36
L-10 Unnecessary safeIncreaseAllowance when a pool-coin amount is zero	37

L-11 Harvest fee charges twice on the same virtual-price band after a dip and recovery	38
L-12 MorphoVault._setRewardTokens does not reject the vault-share token	39
L-13 Morpho harvest charges a performance fee on recovery of prior losses	40
L-14 Fluid harvest accrual subtraction is unclamped and can theoretically underflow-revert	41
L-15 MorphoVault.stateNav(NO_ALLOCATION) reverts where FluidSupply returns zero	42
L-16 MorphoVault.manualClaim omits the treasury-not-set guard enforced by harvest	43
L-17 Redundant fToken self-approval in _exitFluid accumulates an unused allowance	44
3. About MixBytes	45

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Shift is a DeFi yield-routing protocol that allocates user deposits across whitelisted on-chain strategies and cross-chain bridges through a shared `Container` / `StrategyContainer` / `BridgeAdapter` architecture.

This report consolidates two engagements against the same repositories. The first covered a Curve StableSwap-NG gauge strategy (`CurveGauge`) that supplies liquidity to a stable pool and stakes the LP into the corresponding LiquidityGaugeV6, and a USDT0 cross-chain bridge adapter (`USDT0BridgeAdapter`) built on LayerZero V2's OFT composer pattern; that scope has completed an interim round and two remediation rounds. The second, newly added in this revision, covers two ERC-4626 yield-adaptor strategies built on the shared `StrategyTemplate` — `MorphoVault` , which supplies the underlying asset to a Morpho V2 vault, and `FluidSupply` , which supplies it to a Fluid fToken — together with the `StrategyTemplate` base they both inherit. Each adapter holds a single principal position, harvests and compounds Merkle rewards (Angle for Morpho, Fluid for Fluid), charges a performance fee to the treasury, reports per-state NAV to the container, and supports a normal and an emergency exit lifecycle.

The two scopes are presented in their own subsections below.

USDT0 Bridge & Curve Gauge

The audit covered two source files using manual line-by-line review, cross-protocol verification against the upstream Curve `stableswap-ng` and LayerZero V2 source repositories, AI-assisted candidate generation, multi-stage exploit modelling, and end-to-end state-transition tracing. Where the code under review depended on out-of-scope parent contracts or third-party protocols, the relevant call paths were traced into the dependency to confirm or reject the invariants the in-scope code relies upon.

Attack vectors that produced findings included cross-chain message authentication and envelope decoding, OFT-adaptor mainnet/non-mainnet token semantics, source-chain compose injection, the parent-adaptor retry and cache surface, virtual-price-driven fee math, integration with the LiquidityGaugeV6 reward and emission paths, state-machine integrity around `emergencyExit` / `acceptNav` / `reenterToState` , residual accounting after partial exits, and assorted input validation gaps in initialisation and configuration entrypoints.

The following attack vectors were investigated and dismissed. Reentrancy at the strategy and adaptor entrypoints is sufficiently mitigated by the parent's `nonReentrant` guards covering enter, exit, harvest, `emergencyExit`, bridge, `retryBridge`, and claim. LayerZero compose message replay is prevented by the Endpoint's `composeQueue` hash mechanism for the same guid. Decimal-conversion drift inside `IOFT.send` is a no-op for the current 6/6 USDT0 configuration. MEV sandwich on the harvest's inner `add_liquidity` is uneconomic against plain StableSwap-NG pools given the tight curve and 0.04% swap fee. Initialisation front-running is blocked by `_disableInitializers` in both constructors. Storage-layout compatibility with the parent's `__gap` reservations was verified, as was ABI compatibility between imported interfaces and the actual Vyper/Solidity sources at the pinned submodule commits. The `safeIncreaseAllowance` path does not accumulate unbounded grants because allowances are consumed exactly by the immediate downstream call in the happy path. No strict-equality balance traps exist in the audited contracts. The `reenterToState` / `acceptNav` state-transition combinations that reach non-target configurations are recoverable through subsequent transitions under the parent's design, with the relevant NAV blind spots captured separately as findings.

Out of scope for this engagement: third-party protocols (Curve StableSwap-NG and LiquidityGaugeV6, LayerZero V2 OFT and Endpoint, the canonical USDT0 OFT/OFTAdapter contracts, OpenZeppelin libraries), other Shift strategies not listed in scope (`AaveV3Supply` , `CompoundV3Supply` , `FluidSupply`), and any test/mocks/dependency-stub interface files.

The audit identified one critical source-chain compose-injection path that lets any USDT0 holder credit themselves arbitrary amounts on the destination adapter, two high-severity defects in the bridge integration (one in `LzCompose` authentication and envelope decoding, addressed by the client before report delivery; one in the outbound bridging flow where the OFT receiver is the peer container rather than the peer adapter), two medium-severity issues (the parent `retryBridge` mechanism, and a stale harvest snapshot that over-charges successive harvests and eventually blocks enter/exit), and several lower-severity issues in the Curve harvest accounting, partial emergency-exit residuals, and assorted initialization input validation. The recurring recommendation across both contracts is to strengthen source authentication at every cross-chain entrypoint, refresh internal snapshots after every treasury transfer, and disable the redundant sender-side retry mechanism that LayerZero V2's destination-side retry already supersedes.

Following the interim report the client delivered remediation commits — `6820a6d` (USDT0 adapter) and `3c76ca0` (CurveGauge) — which were re-audited against the original findings. All Critical, High, and Medium findings are fixed, together with six of the ten low-severity findings; the remaining four are acknowledged (uncollected CRV/Minter emissions, fee not charged on LP parked outside the gauge state, dormant pool coins excluded from NAV, and the uninitialised-snapshot fee-on-principal path). The re-audit raised one further low-severity finding — the harvest fee can double-charge a virtual-price band after a dip and recovery (no high-water mark) — bringing the low count to eleven. A second remediation round (`2a1d8c2`) then fixed two of these — the uninitialised-snapshot fee-on-principal path (Low #8, via a baseline-unset guard) and the virtual-price double-charge (Low #11, via a high-water mark) — leaving three low-severity findings acknowledged. Per-finding status and verification notes are in Section 2.

Beyond the individual findings, the audit team noted several architectural and implementation choices that are not bugs in themselves but reduce the security envelope of the protocol as a whole. The breadth of privileged roles inherited from the parent platform — `DEFAULT_ADMIN_ROLE` , `BRIDGE_ADAPTER_MANAGER_ROLE` , `CACHE_MANAGER_ROLE` , `OPERATOR_ROLE` , `HARVEST_MANAGER_ROLE` , `EMERGENCY_EXECUTOR_ROLE` , `EMERGENCY_MANAGER_ROLE` , and the reshuffling-related roles — each unlock distinct capabilities; the practical safety of the system depends on careful separation of these roles across operationally-isolated multisigs or DAO governance rather than on the code itself. The parent's `acceptNav(stateId)` design allows the emergency manager to close a resolution into any registered state, a deliberate flexibility choice that in conjunction with the per-state `stateNav` view creates a window during which vault NAV under-reports the strategy's true holdings. Retaining the sender-side `retryBridge` in a LayerZero-backed adapter — where the canonical retry path already lives on the destination chain — adds attack surface the cross-chain protocol does not require. The strategy's `height` -based state machine is powerful and flexible but carries NAV blind spots whenever holdings straddle two states, making strict operator discipline during partial exits a safety prerequisite rather than a recommendation. None of these are individually exploitable in the absence of operator error or external compromise, but together they place a significant share of the security guarantee on operational practice rather than code-enforced invariants.

Morpho & Fluid Strategies

The Morpho and Fluid adapters were reviewed by manual line-by-line analysis of the two leaf contracts and the shared `StrategyTemplate` base, cross-checked against the pinned upstream Morpho V2 and Fluid sources to confirm the share-accounting, exchange-price, and redemption-failure assumptions the in-scope code relies on, supported by AI-assisted candidate generation and end-to-end state-transition tracing across the enter / exit / harvest / emergencyExit / acceptNav lifecycle. Where behaviour depended on the out-of-scope `StrategyContainer` , the relevant container facts were confirmed directly against its source — in particular that the container invokes `harvest()` before every enter and exit, which places the harvest path on the critical path of deposits and withdrawals.

Apart from the checklist of common attack classes (ERC-4626 share-price inflation, reentrancy, proxy initialization, etc.) the following broader, strategy-specific vectors were examined:

- **Reward harvesting and compounding** — whether harvested rewards are converted and reinvested without value leakage or stranded balances, given that the adapters swap the full reward balance through the container router and silently skip unroutable tokens with no sweep.
- **Harvest coupling to the user surface** — whether harvest liveness gates deposits and withdrawals, given that the container invokes `harvest()` before every enter and exit and the reinvest leg deposits the whole balance without

isolation.

- **Performance-fee accounting** — whether fees accrue only on realized net yield, given that the appreciation checkpoint, the reinvest-fee leg, donations, and loss-then-recovery sequences all feed the same fee computation.
- **NAV computation and per-state consistency** — whether `stateNav` stays faithful and readable across the whole lifecycle, including the empty and mixed-holding states, given that it is derived from the upstream vault's `convertToAssets` and the container oracle.
- **Privileged-role scope and least privilege** — whether a limited role (Merkle claimer, harvest manager, reshuffling executor, emergency executor) can act beyond its remit, such as moving principal, changing custody posture, or weakening slippage protection.
- **Emergency-exit and NAV-resolution lifecycle** — whether the `emergencyExit` / `acceptNav` / `reenterToState` paths on the shared `StrategyTemplate` can strand funds, mis-report NAV, or block recovery (the base-contract issues are reported in the platform audit).
- **Permissionless interactions** — whether token donations or direct distributor claims can shift value between cohorts, force a treasury fee on principal, or leave off-list tokens stranded.

The following are architectural and trust characteristics noted during the review; they are not vulnerabilities in themselves but shape the system's risk surface:

- The reward-compounding path is deliberately best-effort (`minAmountOut = 0` , `mustSucceed = false`): it keeps a single bad route from blocking the harvest, but provides no execution-price protection and silently leaves an unroutable reward token idle on the strategy, with no sweep or rescue primitive in scope.
- A direct donation of the underlying or of the vault share inflates the reported NAV and is folded into the next harvest's accrual and fee'd as if it were yield; this is donor-funded and not attacker-profitable, but it lets a third party move value onto the existing cohort and force a treasury fee on principal.
- The two adapters are deliberately asymmetric because a Morpho reward can itself be the underlying asset while a Fluid reward cannot; this is the root of the Morpho manual-claim reinvest path and the extra reward-token-setter surface, and the Fluid manual-claim correspondingly omits the NAV-resolution-mode guard the Morpho path carries (harmless, as the Fluid claim touches no NAV, checkpoint, or share balance).
- Each adapter holds a single principal position and mints no shares of its own, so its reported value is entirely a function of the upstream vault's `convertToAssets` and the container's oracle — the correctness of NAV is inherited from those out-of-scope dependencies.
- `harvest()` sits on the critical path of every enter and exit (container coupling), so harvest liveness is, in effect, deposit and withdrawal liveness: any condition that reverts a harvest also freezes the user surface.
- The underlying vault's deposit cap and the Fluid `fToken`'s monotonic exchange price are external invariants the in-scope code relies on but does not enforce; a change in either upstream surfaces as a harvest revert rather than a handled condition.
- All value-bearing tokens (underlying, reward, output) are drawn from the container whitelist; the strategy never accepts an arbitrary token address from an unprivileged caller.
- The shared `StrategyTemplate` places a meaningful share of the safety envelope on operational discipline — a broad set of inherited privileged roles, an `acceptNav(stateId)` that lets the emergency manager close a resolution into any registered state, and a height-based state machine with NAV blind spots whenever holdings straddle two states during a partial exit; the base-contract findings are reported in the platform audit.

Out of scope for this part of the engagement: the Angle and Fluid Merkle distributors, the underlying Morpho V2 vault and Fluid `fToken` implementations, OpenZeppelin libraries, and the other Shift strategies not listed in scope.

The review identified no critical or high-severity issue and no unprivileged theft or principal-loss path. It produced three medium-severity findings — reward compounding at a zero slippage floor, the Merkle-claimer principal re-exposure after an emergency exit, and the harvest reinvest revert that blocks deposits and withdrawals — and six low-severity findings. All are newly reported in this revision and are recommended to be fixed, though none blocks a safe deployment on its own. Overall the Morpho and Fluid adapters are well-constructed and defensively coded — consistent `nonReentrant` guards, checks-

effects-interactions ordering, `balanceOf` -delta accounting, `SafeERC20` throughout, and set-once immutables — and the issues identified are availability, value-leakage, and accounting-precision refinements rather than fundamental design flaws.

The latest Morpho/Fluid remediation round resolved seven of the nine newly reported findings in that scope. The two remaining medium-severity findings are acknowledged by the client, and the reviewed remediation diff did not introduce new reportable vulnerabilities.

1.3 Project Overview

Summary

CLIENT	Shift
CATEGORY	Yield Aggregator
PROJECT	USDT0 Bridge & Strategies
TYPE	Solidity
PLATFORM	EVM
TIMELINE	26.05.2026 — 08.07.2026

Scope of Audit

[contracts/curve-gauge/CurveGauge.sol](#)
[contracts/USDT0BridgeAdapter.sol](#)
[contracts/morpho/MorphoVault.sol](#)
[contracts/fluid/FluidSupply.sol](#)

Versions Log

DATE	COMMIT HASH	NOTE
26.05.2026	2da1e5593b2231802b7cfc189fb5be585a1b47df	Strategies initial commit
26.05.2026	61492205f0a535fbcdf7f2d8d788fb7472fc4850	Bridge-adapter-usdt0 initial commit
28.05.2026	a677f7ab272e6075530646d776778b525b484b94	Bridge-adapter-usdt0 follow-up fix
17.06.2026	6820a6dd93ed81af310b38b665ef1c4427bacdc8	Bridge-adapter-usdt0 re-audit fixes
17.06.2026	3c76ca0287ad3e72826d53c8b4898da67de567a8	Strategies re-audit fixes
24.06.2026	2a1d8c2f8ca83f7f6efe25d06ad83aedace85401	Strategies re-audit fixes #2
25.06.2026	b6e80bf0c9939d4a0bf54b8b0a66df387e2893fb	Strategies Morpho/Fluid interim scope commit
07.07.2026	462f7817b4f09b5b443e5f2214450f0c63bea500	Strategies Morpho/Fluid re-audit fixes

Mainnet Deployments

At this stage, the contracts have not yet been deployed. Verification that the deployed contracts correspond to the audited code will be conducted once the client provides the addresses of the deployed contracts.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

ID	FINDING	SEVERITY	STATUS
C-1	Source-chain compose injection on the destination adapter	● CRITICAL	● FIXED
H-1	IzCompose rejects the LayerZero Endpoint and misreads the OFT envelope	● HIGH	● FIXED
H-2	Outbound bridge sends USDT0 to the peer container instead of the destination adapter	● HIGH	● FIXED
M-1	Inherited retryBridge creates unnecessary replay and payload-substitution surface	● MEDIUM	● FIXED
M-2	lastStoredGaugeBalance snapshot not refreshed after treasury transfers in _harvest	● MEDIUM	● FIXED
M-3	Reward compounding swaps the full reward balance at a zero slippage floor	● MEDIUM	● ACKNOWLEDGED
M-4	Merkle claimer can redeposit parked Morpho principal outside the recorded state	● MEDIUM	● FIXED
M-5	Harvest reinvest leg reverts at the vault deposit cap, blocking deposits and withdrawals	● MEDIUM	● ACKNOWLEDGED
L-1	CRV emissions are never claimed and non-pool-coin reward tokens are left idle	● LOW	● ACKNOWLEDGED
L-2	Harvest fee underflows on virtual_price decrease, blocking deposits and withdrawals	● LOW	● FIXED

ID	FINDING	SEVERITY	STATUS
L-3	payload.refundRecipient and payload.gasLimit are not validated	● LOW	● FIXED
L-4	Harvest fee misses virtual-price growth accrued outside the gauge state	● LOW	● ACKNOWLEDGED
L-5	Initialization does not verify cross-field consistency	● LOW	● FIXED
L-6	Dormant pool coins are excluded from _curveGaugeNav and _curveLpNav	● LOW	● ACKNOWLEDGED
L-7	Math.mulDiv reverts when ICurveStableSwapNG.totalSupply() == 0	● LOW	● FIXED
L-8	Uninitialised lastStored* charges fee on principal	● LOW	● FIXED
L-9	Compose payload encodes instruction.amount instead of the OFT-delivered amount	● LOW	● FIXED
L-10	Unnecessary safeIncreaseAllowance when a pool-coin amount is zero	● LOW	● FIXED
L-11	Harvest fee charges twice on the same virtual-price band after a dip and recovery	● LOW	● FIXED
L-12	MorphoVault._setRewardTokens does not reject the vault-share token	● LOW	● FIXED
L-13	Morpho harvest charges a performance fee on recovery of prior losses	● LOW	● FIXED
L-14	Fluid harvest accrual subtraction is unclamped and can theoretically underflow-revert	● LOW	● FIXED
L-15	MorphoVault.stateNav(NO_ALLOCATION) reverts where FluidSupply returns zero	● LOW	● FIXED
L-16	MorphoVault.manualClaim omits the treasury-not-set guard enforced by harvest	● LOW	● FIXED

ID	FINDING	SEVERITY	STATUS
L-17	Redundant fToken self-approval in _exitFluid accumulates an unused allowance	● LOW	● FIXED

2.1 Critical

C-1 Source-chain compose injection on the destination adapter

● CRITICAL

● FIXED AS OF 6820a6dd

DESCRIPTION

The destination adapter's `lzCompose` at `contracts/USDT0BridgeAdapter.sol:125-136` authenticates two facts about the incoming call:

1. `msg.sender == lzEndpoint` confirms the LayerZero Endpoint is the immediate caller.
2. `approvedOApps[_fromOApp]` requires the originator of the destination-chain `sendCompose` call to be allow-listed.

Under LayerZero V2 OFT compose semantics, `_fromOApp` is the address that called `endpoint.sendCompose` on the destination chain. For the OFT compose path that caller is always the local destination OFT (it issues `sendCompose` from inside its own `_lzReceive`). The check therefore does not filter on source-chain identity.

The function never reads `composeFrom` or `srcEid` from the wrapped envelope, and never cross-checks the inner `amount` against the OFT-delivered `amountReceivedLD`. The `(claimer, amount)` pair encoded by the source-side caller is taken as authoritative and forwarded directly to `_finalizeBridge`. An inbound compose should credit `claimableAmounts[claimer][token]` only if it originated from a Shift-controlled outbound flow on a known source chain; any other source-chain caller — including any address holding USDT0 — should be rejected before reaching `_finalizeBridge`.

Concrete attack flow:

1. The attacker holds 1 USDT0 on any chain where the USDT0 OFT has a peer pointing at the destination chain.
2. The attacker calls `IOFT.send` directly with `sendParam.to = bytes32(destinationAdapter)`, a small `amountLD`, and `composeMsg = abi.encode(attacker, X)` where `X` is an arbitrary inflated amount.
3. The source OFT debits 1 USDT0 from the attacker; the LayerZero message is dispatched.
4. On the destination chain the OFT's `_lzReceive` credits the destination adapter with 1 USDT0 (the real delivered amount), then queues the compose. The Executor invokes the Endpoint, which calls the destination adapter's `lzCompose` with `_fromOApp = localOft` and the wrapped message containing `(attacker, X)`.
5. Both checks in `lzCompose` pass. `_finalizeBridge(attacker, usdt0, X)` credits `claimableAmounts[attacker][usdt0] += X`.
6. The attacker calls `claim(usdt0)` and withdraws `X` USDT0 from the adapter's balance — the pool that backed legitimate claimers is now short by `X - 1`.

The attack is repeatable per source chain. The marginal cost per attack is one USDT0 plus the LayerZero `nativeFee`; the recoverable amount is bounded only by the adapter's outstanding USDT0 balance, which grows as legitimate inbound bridges accumulate.

The bug is reachable by any address holding USDT0 on any source chain and a single successful execution withdraws funds belonging to legitimate inbound claimers without compensating top-up. The result is direct theft of unclaimed user funds from the destination adapter, repeatable across nonces and source chains, with no on-chain recovery primitive other than a proxy upgrade — which is why we classify it as a Critical severity issue.

RECOMMENDATION

We recommend extracting `srcEid` and `composeFrom` from the OFT envelope inside `lzCompose` and validating both against a per-source allow-list of Shift's outbound adapter addresses on each supported source chain. As defence in depth, the credited amount should be cross-checked against the OFT-delivered amount carried in the same envelope. The local-OFT-identity check that currently exists can be reduced to a single `require(_from0App == oft)` invariant.

2.2 High

H-1 `lzCompose` rejects the LayerZero Endpoint and misreads the OFT envelope

● HIGH

● FIXED AS OF a677f7ab

DESCRIPTION

The destination-chain entrypoint `USDT0BridgeAdapter.lzCompose` at :118–129 is the function that the LayerZero V2 stack invokes after each inbound bridge to credit a claimer's USDT0 balance. The function carries two independent defects that together make the entire inbound path non-functional and strand every inbound USDT0 transfer on the adapter until a proxy upgrade replaces the implementation.

The caller-authentication line uses `require(msg.sender == oft, NotOFT())`. Under LayerZero V2 the external call into a composer is issued by the Endpoint, not by the OFT — the OFT is passed as `_from` rather than appearing as `msg.sender`. The `require` therefore reverts on every legitimate delivery. The Endpoint catches the revert; the `composeQueue` hash is rolled back to its pre-call state so the message can be retried after the implementation is replaced, but until then every inbound bridge is dead on arrival.

The decode line uses `abi.decode(_message, (address, uint256))` directly on the wrapped message that the LayerZero stack delivers. Per the OFT compose codec, the destination OFT wraps the source-side payload with a 76-byte header `[nonce 8B][srcEid 4B][amountLD 32B][composeFrom 32B]` before submitting it to `endpoint.sendCompose`. Decoding the wrapped buffer as a tuple of `(address, uint256)` reads the nonce slice as an address; Solidity 0.8.x's upper-bits cleanliness check for `address` rejects this for any non-zero nonce. Even if the caller-authentication line were corrected in isolation, this decode would still abort the call.

Combined effect: on every inbound bridge the OFT credits the adapter with the bridged amount via `_credit` on the destination chain, but `lzCompose` reverts before reaching `_finalizeBridge`. The claim mapping is never incremented, and the subsequent `claim(usdt0)` reverts with `ZeroAmount`. The contract is deployed behind an upgradeable proxy and the `composeQueue` hashes are preserved across the failed deliveries, so the recovery path is an implementation upgrade followed by replay of the queued composes.

`decodeLzComposeMessage` at :82–84 carries the same envelope-versus-inner-payload mismatch and would have to be corrected in tandem.

The bug fires on every legitimate inbound delivery and removes the only on-chain credit primitive for inbound USDT0, locking funds on the destination adapter until an implementation upgrade ships and the queued composes are replayed. The funds are recoverable through an upgrade and replay rather than lost outright — the High classification reflects that the path is universally broken but the remediation does not require off-chain custody recovery.

RECOMMENDATION

We recommend authenticating the LayerZero Endpoint as the immediate caller, anchoring the originator on the local OFT through `_fromOApp`, and stripping the OFT envelope through the canonical compose codec before decoding the inner payload. The Endpoint address should be supplied at initialisation alongside cross-field consistency checks against the OFT's reported

endpoint. Pre-mainnet integration tests should exercise **LzCompose** end-to-end against a forked LayerZero Endpoint so the inbound path is covered by CI.

H-2 Outbound bridge sends USDT0 to the peer container instead of the destination adapter

● HIGH

● FIXED AS OF 6820a6dd

DESCRIPTION

For a correct OFT compose integration, `SendParam.to` must carry the destination adapter address so both the token credit and the compose delivery target the adapter; the inner payload then carries the claimer identity for `_finalizeBridge`.

In `USDT0BridgeAdapter._bridge` at :146-164 the function signature declares three parameters but names only the first two. The third parameter is the destination peer that the parent's `BridgeAdapter.bridge` passes in as `peers[instruction.chainTo]` — the configured destination-chain adapter address. The function ignores this argument and instead builds `SendParam.to` from the `receiver` argument inside `quoteBridgeNativeFee` at :97.

The `receiver` argument is supplied by the caller of `bridge()`. Both upstream callers in the parent platform — `ContainerAgent._bridgeToken` and `ContainerPrincipal._bridgeToken` — pass `peerContainer` as that argument. The resulting `SendParam.to` is therefore the peer container's address rather than the destination adapter's address.

This produces three downstream effects on the destination chain:

1. The destination OFT's `_lzReceive` credits the peer container directly with USDT0 (the `_credit` step uses `sendTo` from the wrapped LayerZero message).
2. The compose message is queued for delivery to the peer container. The container contracts do not implement `lzCompose`, so the Executor's attempted invocation produces an `LzComposeAlert` rather than a successful credit.
3. The destination adapter's `lzCompose` is never invoked; `claimableAmounts[claimer][usdt0]` stays at zero.

The peer container's subsequent attempt to advance its accounting through `IBridgeAdapter.claim(usdt0)` reverts inside the adapter's `_claim` because the claim mapping is zero. The container's `claimCounter` does not decrement, the bridge status does not advance to `BridgeClaimed`, and the deposit or withdrawal flow that triggered the bridge remains stuck. The USDT0 is physically resident on the peer container but is not visible to the container's accounting layer.

The bug fires on every legitimate outbound bridge and breaks the cross-chain accounting path end-to-end. The USDT0 is physically present on the peer container so principal is not directly lost, but the container's state machine is jammed until a contract upgrade restores the destination credit flow, and the cross-chain deposit and withdrawal workflows remain frozen in the interim. The High classification reflects that every cross-chain operation is affected with no operator workaround short of an upgrade.

RECOMMENDATION

We recommend using the third argument of `_bridge` (the configured destination peer) as the OFT `to` field, and keeping the encoded payload's claimer field as the final beneficiary credited by `_finalizeBridge`. End-to-end tests of the full container ↔ adapter ↔ OFT round-trip — initiating a bridge from the source container, exercising the destination adapter's `lzCompose`, and finalising the claim — would catch this category of integration drift before deployment.

2.3 Medium

M-1 Inherited `retryBridge` creates unnecessary replay and payload-substitution surface

● MEDIUM

● FIXED AS OF 6820a6dd

DESCRIPTION

The parent's `retryBridge` at `shift-defi-platform/contracts/BridgeAdapter.sol:143-159` re-invokes the adapter's `_bridge` hook to re-send a cached bridge instruction. Inherited as-is by `USDT0BridgeAdapter._bridge`, this path carries several compounding defects that together make any retry invocation unsafe on a LayerZero-backed adapter.

LayerZero V2 already provides destination-side retry: when delivery on the destination chain fails or stalls, anyone can re-execute by calling the same Endpoint method with the original parameters. The packet remains in the inbound queue until successfully delivered. A sender-side retry path should therefore be unnecessary for any normal failure mode. Once `IOFT.send` on the source has returned, the packet is committed to LayerZero's DVN/Executor pipeline; a second call to `_bridge` produces a new LayerZero message with a new nonce, which constitutes a duplicate delivery rather than a retry of the original.

The cache key built by `retryBridge` covers `(token, chainTo, amount, receiver, nonce)` but does not bind to `instruction.payload`. The same tuple of cached fields therefore matches any payload, and a retry can submit a fabricated payload alongside a legitimate cached tuple. For this adapter the payload carries the destination compose fields `(dstEid, claimer, refundRecipient, gasLimit)`, which means a retry can redirect the destination credit to any claimer the caller chooses while passing the cache existence check.

`retryBridge` re-runs `_bridge` without `safeTransferFrom` from the caller. The adapter's own balance funds the resulting `IOFT.send`. On the audited deployment the adapter does not normally hold USDT0 between cross-chain operations, so a retry of a nonce that already delivered reverts with insufficient balance and the immediate damage is bounded. The structural concern, however, is that any USDT0 the adapter does accumulate from any source — operator top-up, accidental transfer, future routing changes — becomes spendable by a retry of a previously-delivered nonce without re-checking the corresponding cached claim accounting.

The retry path is gated by `CACHE_MANAGER_ROLE`, which is a trusted role and reduces the likelihood that any of these surfaces is exercised. The consequences of a single honest misclick or a key compromise, however, range from duplicate cross-chain delivery to redirection of cached value to an attacker-controlled claimer. The Medium classification reflects that an active trigger by a privileged role is required while the impact of that trigger is severe, and that the cleanest mitigation is to remove the path entirely rather than patch each individual surface.

RECOMMENDATION

We recommend disabling `retryBridge` at the leaf by overriding the inherited function so that it reverts unconditionally. The recovery primitives for stuck cross-chain delivery should be the destination-chain Endpoint methods. If a narrower recovery path is ever required for the rare case where a source-side send succeeded but the packet never reached destination commit, we

recommend exposing a separate role-gated entry point that requires an off-chain proof of non-commitment rather than relying on cache existence. Independently, we recommend tightening the cache key in the parent `BridgeAdapter` so that `instruction.payload` is bound, which removes the payload-substitution surface for any future adapter that does choose to retain a sender-side retry path.

M-2 `lastStoredGaugeBalance` snapshot not refreshed after treasury transfers in `_harvest`

● MEDIUM

● FIXED AS OF 3c76ca02

DESCRIPTION

The snapshot variables `lastStoredGaugeBalance` and `lastStoredVirtualPrice` are written inside `_enterCurveGauge` at :130-133 immediately after `gauge.deposit`. The snapshot represents the strategy's gauge holdings as of the most recent state-mutating action against the gauge, and is the basis for the virtual-price-driven fee computed at the top of `_harvest`. Any treasury transfer that follows a snapshot write must update the snapshot to preserve that invariant.

Two paths inside `_harvest` violate the invariant:

The transfer at :233 sends `vars.gaugeTokensToTreasury` to the treasury, after which `claim_rewards()` runs at :240. If the post-claim balance deltas of `underlyingAsset0` and `underlyingAsset1` are both zero, the function early-returns at :245-247 without refreshing the snapshot. The snapshot remains greater than the actual on-chain gauge balance by the transferred amount.

The transfer at :264 sends `gaugeTokensToTreasury` after the rewards-leg `_enterCurveGauge()` at :259 has written the snapshot to the post-stake balance. The snapshot remains greater than the actual on-chain gauge balance by the transferred amount.

On the next harvest the numerator `currentGaugeBalance × currentVirtualPrice - lastStoredGaugeBalance × lastStoredVirtualPrice` is computed as if the previously-transferred amount had not been taken. The `lpValueDelta` therefore includes the previously-charged fee in its base, and the treasury is charged again on top of that base. For a concrete trace: starting at gauge balance 1000, virtual price 1.000, and fee fraction 10%, a first harvest after 1% virtual-price growth charges approximately 0.99 to the treasury and leaves the snapshot at (1000, 1.000) while the gauge balance drops to 999.01; a second harvest after another 1% virtual-price growth computes $999.01 \times 1.020 - 1000 \times 1.000 = 18.99$, charges approximately 1.86 to the treasury, where the ideal accrual for the second interval was approximately 0.98 — the second-harvest charge is roughly twice the value that the cycle actually generated. Each subsequent no-rewards harvest compounds the drift, the per-call over-charge grows with the cumulative untracked transfer amount, and the difference is taken from depositors' yield in favour of the treasury.

In the limit where the cumulative untracked transfer exceeds the virtual-price growth between calls, the numerator flips negative and the unchecked subtraction underflows. Because deposits and withdrawals auto-invoke harvest through the parent's enter/exit hooks, the resulting revert blocks the user surface until an emergency exit refreshes the snapshot through `_exitCurveGauge`.

Under the planned plain-pool deployment, `claim_rewards()` returns nothing when the gauge has no registered extra reward tokens, making the no-rewards early-return the common execution path. Snapshot staleness therefore accumulates on every harvest until a deposit or withdrawal refreshes it through `_enterCurveGauge` or `_exitCurveGauge`. In operating regimes where harvests are more frequent than user actions, the compounding drift can outpace the cycle's actual yield, causing the virtual-price-delta computation to go negative and reverting the user-facing surface of the vault.

RECOMMENDATION

We recommend refreshing the snapshot after every treasury transfer in the harvest path. Equivalently, the harvest can subtract the transferred amount from the in-memory snapshot variables before storing them. Either approach restores the invariant that the snapshot reflects the strategy's actual gauge holdings at the end of the harvest.

M-3 Reward compounding swaps the full reward balance at a zero slippage floor

● MEDIUM

● ACKNOWLEDGED

DESCRIPTION

Both strategy adapters compound their reward tokens by swapping the full reward balance to the underlying asset inside `_harvest`, passing a zero minimum-output floor. In `MorphoVault._harvest` at :240-243 the reward loop calls `_swapToInputTokens(rewardTokens[i], underlyingAsset, 0, false)` for every configured reward token, and in `FluidSupply._harvest` at :220-224 the Merkle reward token is swapped the same way.

The shared helper `StrategyTemplate._swapToInputTokens` at :621-636 reads the entire `balanceOf(tokenIn)`, approves it to the container's `SwapRouter`, and forwards the caller-supplied `minAmountOut` to `tryPredefinedSwap`. The router is itself capable of enforcing a floor — `SwapRouter.swap` reverts unless `deltaTokenOut >= minAmountOut` — but because the adapters pass `0`, that check degenerates to `deltaTokenOut >= 0`, which always holds. The reward swap therefore executes at whatever price the route returns.

A swap of the whole accumulated reward balance at a zero floor on a public route is fully exposed to MEV: a searcher can sandwich the harvest transaction and capture the spread between the route's mid-price and the realized execution price. Because harvest is invoked on a schedule and on every enter and exit, the swap is observable and orderable in the mempool. The extracted value is taken from the position's compounded yield on every harvest where a reward balance exists and a public route is configured.

A second consequence of the `mustSucceed = false` flag is that a reward token with no predefined route causes `tryPredefinedSwap` to return `false`, which is silently ignored. Such a token — a route-less reward, a de-listed reward, or an off-list token pushed onto the strategy — is never converted and accumulates on the strategy indefinitely; there is no sweep or rescue primitive in scope, so the value is stranded.

The loss is bounded to reward value rather than principal, and the realized magnitude depends on reward size and route liquidity, which is why we classify it as Medium. The structural problem is that the strategy disables the only slippage protection available to it by hard-coding a zero floor.

RECOMMENDATION

We recommend not performing the reward swap inside the strategy at all. Instead, the harvested reward tokens should be returned to the container, allowing the container to swap them at a time and price of its choosing and reinvest the proceeds back into the strategy through the normal enter path; this also removes the route-less-token strand because the tokens leave the strategy with the container. If in-strategy swapping is retained, the adapters should pass an operator-specified non-zero `minAmountOut` (and a deadline) through the harvest call so the router's existing floor check becomes effective.

CLIENT'S COMMENTARY

Acknowledged and will be fixed in a further upgrades. For now private node will be used for MEV protection.

M-4 Merkle claimer can redeposit parked Morpho principal outside the recorded state

● MEDIUM

● FIXED AS OF 462f7817

DESCRIPTION

After a full emergency exit, the Morpho strategy parks its entire position as raw underlying and records `UNDERLYING_ASSET_STATE_ID` as the current state with NAV-resolution mode cleared. From that state, `MorphoVault.manualClaim` at `:266-321` — callable by the `MERKLE_CLAIMER_ROLE` — can move the entire parked balance back into Morpho.

`manualClaim` guards only on `!isNavResolutionMode()` (`:272`), which passes because a full `emergencyExit` finalizes resolution. It then claims from the Angle distributor and, for any claimed token equal to the underlying asset, calls `_enterMorphoVault()`:

```
for (uint256 i = 0; i < vars.rewardTokensLength; i++) {
    if (tokens[i] == vars.underlyingAssetCached) {
        _enterMorphoVault();
        break;
    }
}
```

`_enterMorphoVault` at `:157-173` deposits the strategy's entire underlying `balanceOf` — not the newly claimed delta — into the Morpho vault, and does not update `_currentStateId`. The deterministic result on the post-emergency state is threefold:

1. The strategy's full parked principal is re-deposited into the very protocol the emergency exit removed it from, triggered by a reward-only role. Nothing constrains `manualClaim` to the recorded allocation state, and when the recorded state is the underlying state the strategy should not re-enter the vault at all.
2. `_currentStateId` remains `UNDERLYING_ASSET_STATE_ID`, so `stateNav(currentState)` values the now near-zero underlying balance while the real value sits in unaccounted Morpho shares — the reported NAV no longer reflects the holdings.
3. The lifecycle cannot reach those shares from the underlying state: a normal `enter` does not target the vault from this state and `exit` only handles the token leg, so recovery requires an `EMERGENCY_MANAGER` to `reenterToState(MORPHO_VAULT_STATE_ID)`.

The trigger is the limited `MERKLE_CLAIMER_ROLE` and additionally requires a valid strategy-bound Angle claim denominated in the underlying asset, and the funds remain in the approved vault (recoverable by the emergency manager), which is why we classify it as Medium rather than higher. The defect is the absence of a state constraint on a custody-changing helper reachable from a reward-only role.

RECOMMENDATION

We recommend gating the reinvestment in `manualClaim` on the recorded allocation state — reinvest only when `_currentStateId == MORPHO_VAULT_STATE_ID`, and otherwise leave the claimed underlying idle for the emergency and allocation roles to handle.

M-5 Harvest reinvest leg reverts at the vault deposit cap, blocking deposits and withdrawals

● MEDIUM

● ACKNOWLEDGED

DESCRIPTION

The parent container invokes `harvest()` before every enter and exit on the strategy, so any revert inside `_harvest` blocks not only harvesting but the entire deposit and withdrawal surface of the strategy. Both adapters reinvest idle underlying back into the underlying vault inside `_harvest` without isolating that call from a failure.

In `MorphoVault._harvest` at :244-247:

```
vars.lpAmountBefore = IERC4626(vars.morphoVaultCached).balanceOf(address(this));
_enterMorphoVault();
vars.reinvestLpDelta = IERC4626(vars.morphoVaultCached).balanceOf(address(this)) -
vars.lpAmountBefore;
```

`_enterMorphoVault` deposits the strategy's whole underlying `balanceOf` via `IERC4626.deposit`, and `FluidSupply._enterFluid` does the same for the fToken. ERC-4626 `deposit` reverts when `assets > maxDeposit(receiver)`. When the underlying vault is at its supply cap — a Morpho market at its per-market cap, or a paused / capped Fluid fToken — and there is idle underlying to reinvest (for example reward tokens just swapped into underlying earlier in the same harvest), the deposit reverts, the whole `_harvest` reverts, and because `harvest()` precedes `exit()` user withdrawals are blocked for as long as the vault remains full.

The supply cap is reachable by ordinary third-party deposits into the underlying vault, so the trigger is external market state rather than operator action. No funds are lost (the call reverts atomically) and the condition self-clears when the vault drops below its cap, but withdrawals are frozen while it holds, which is why we classify it as Medium.

RECOMMENDATION

We recommend wrapping the reinvest leg (`_enterMorphoVault` / `_enterFluid`) inside `_harvest` in a `try/catch` so that a failed reinvest is skipped — leaving the underlying idle for the next cycle — rather than reverting the whole harvest and, with it, the deposit and withdrawal paths. Bounding the deposit at `min(balanceOf, maxDeposit(receiver))` is a complementary hardening.

CLIENT'S COMMENTARY

Acknowledged. Proposed fix does not entirely prevent pause in withdrawal processing due to inability to process deposit batch when target protocol reaches deposit cap. Our risk system detects approaching deposit cap and executes reshuffling preventively.

2.4 Low

L-1 CRV emissions are never claimed and non-pool-coin reward tokens are left idle

● LOW

● ACKNOWLEDGED

DESCRIPTION

`CurveGauge._harvest` at `contracts/curve-gauge/CurveGauge.sol:212-266` calls `ILiquidityGaugeV6.claim_rewards()` and measures the post-claim balance deltas of `underlyingAsset0` / `underlyingAsset1`. Per the Curve gauge implementation at `LiquidityGauge.vy:467-477`, `claim_rewards` sweeps only the "extra reward tokens" registered through `add_reward` / `deposit_reward_token`. Native CRV emissions are tracked in `integrate_fraction[user]` on the gauge and materialise on-chain only when the canonical Curve `Minter` contract is invoked via `Minter.mint(gauge)`. The strategy does not interact with the Minter and does not import the Minter interface.

Additionally, `_exitCurveGauge` calls `gauge.withdraw` without first invoking `claim_rewards`. The pre-withdraw checkpoint on the gauge preserves accrued extras in `claim_data` so nothing is lost at the gauge layer, but those amounts sit on the gauge contract uncollected until a subsequent strategy action reaches `claim_rewards` — which `_harvest` itself early-returns for any state other than `CURVE_GAUGE_STATE_ID`. If the lifecycle proceeds through `emergencyExit` into `UNDERLYING_ASSETS` and never re-enters the gauge state, accrued extras remain idle on the gauge.

A second-order effect is that extra reward tokens whose address does not match `underlyingAsset0` or `underlyingAsset1` land on the strategy after `claim_rewards`, but the asset-delta logic does not see them. Those tokens remain idle on the strategy, contributing neither to NAV nor to the treasury fee.

A harvest should work as follows: pull every reward stream the gauge entitles the strategy to (CRV via the Minter, plus all registered extras), normalise non-pool-coin tokens into one of the pool coins, then route the result through the existing fee / compound accounting. The current implementation skips the first half and only partially handles the second.

RECOMMENDATION

We recommend adding a dedicated CRV-claim step to `_harvest` that invokes the canonical `Minter` on chains where one is deployed, and a generic reward-handling step that enumerates the gauge's registered reward tokens and converts non-pool-coin balances into a pool coin before the asset-delta measurement. The Minter address should be supplied via configuration so the same code can run on every supported chain. Pools that do not have a Minter or do not stream extras can be supported with an empty-set early return.

L-2 Harvest fee underflows on virtual_price decrease, blocking deposits and withdrawals

● LOW

● FIXED AS OF 3c76ca02

DESCRIPTION

The harvest fee computation at `contracts/curve-gauge/CurveGauge.sol:226-235` uses an unchecked subtraction:

```
vars.lpValueDelta =  
    (vars.currentGaugeBalance * vars.currentVirtualPrice -  
     lastStoredGaugeBalance * lastStoredVirtualPrice) / vars.currentVirtualPrice;
```

Between two harvests with no enter/exit in between, the gauge share balance is unchanged, so the sign of the subtraction depends solely on whether the pool's `virtual_price` has decreased. Per `CurveStableSwapNG.vy:1740-1755` and the surrounding `_balances` implementation, plain (`asset_type == 0`) pools have a monotonically non-decreasing `virtual_price` under normal use; a rare exception is a governance `ramp_A` down-ramp. Rate-bearing pool types (oracle or ERC-4626) can also see `virtual_price` decreases on rate updates.

Because deposits and withdrawals on the strategy auto-invoke `harvest()` through the parent's enter/exit hooks, an underflow here blocks the entire user surface until an emergency exit is performed. The fee should work as follows: zero or negative virtual-price growth should not produce a fee, and the rewards leg of harvest should still execute irrespective of whether a fee was charged on the first leg.

RECOMMENDATION

We recommend computing the fee-eligible delta with a guarded subtraction that produces zero when growth is non-positive, and ensuring that the reward-claim and compound logic runs unconditionally regardless of the first-leg outcome.

CLIENT'S COMMENTARY

MixBytes: Fixed for the reported case — the guard removes the underflow on a virtual-price decrease and the reward leg now runs independently. Note it guards only the price factor, not the full product, so a future path that reduced the gauge balance below the snapshot without a same-transaction refresh would still underflow; this is not reachable today (the snapshot tracks the balance on every change and the balance cannot be reduced externally), so clamping the subtraction to zero is defensive hardening.

L-3 `payload.refundRecipient` and `payload.gasLimit` are not validated

● LOW

● FIXED AS OF 6820a6dd

DESCRIPTION

`USDt0BridgeAdapter.quoteBridgeNativeFee` decodes the bridger-supplied payload and threads `payload.refundRecipient` into `IOFT.send` and `payload.gasLimit` into the compose option. Neither field is range-checked.

A zero `gasLimit` causes the destination compose execution to revert at the first opcode, leaving the delivered USDt0 stranded on the destination adapter without a corresponding claim credit. A zero `refundRecipient` produces implementation-defined behaviour in the LayerZero Endpoint depending on the pinned version, ranging from outright revert to silent dust burn.

The intended invariant is that an outbound bridge always produces either a normal delivery or a clean revert at the source — never a partial state in which tokens are minted at the destination without a corresponding claim credit. Input validation at the adapter is the natural place to enforce this invariant before the bridger's data flows into the OFT.

RECOMMENDATION

We recommend rejecting zero values for `refundRecipient` and `gasLimit` in `quoteBridgeNativeFee` before the values flow into the LayerZero send path.

L-4 Harvest fee misses virtual-price growth accrued outside the gauge state

● LOW

● ACKNOWLEDGED

DESCRIPTION

`_harvest` early-returns when the current state is not `CURVE_GAUGE_STATE_ID`, so no fee is charged on virtual-price growth while the strategy is parked in another state. The snapshot variables `lastStoredGaugeBalance` and `lastStoredVirtualPrice` advance only inside `_enterCurveGauge` and `_exitCurveGauge`.

When an emergency exit lands the strategy in `CURVE_LP_STATE_ID`, the LP tokens are held directly on the strategy address until a subsequent `reenterToState(CURVE_GAUGE_STATE_ID)` re-stakes them. The pool's `virtual_price` continues to grow during that window as swap fees accrue, and the value of the locally-held LP increases correspondingly. Harvest is a no-op in `CURVE_LP_STATE_ID`, so no fee is taken on that growth, and when the strategy eventually re-enters the gauge state `_enterCurveGauge` overwrites `lastStoredVirtualPrice` to the current pool value. The parking-window growth is permanently outside the fee accounting; depositors capture the entire interim, the treasury captures none.

The gap is small in normal operation where the strategy spends most of its time in the gauge state. It widens whenever operational decisions or a paused gauge park the strategy in `CURVE_LP_STATE_ID` for an extended period.

RECOMMENDATION

We recommend tracking the strategy's total LP-equivalent holdings — gauge-staked plus locally-held LP — as a single cumulative snapshot that advances on every harvest regardless of which state the strategy is currently in. The fee-eligible delta becomes $(\text{totalLpBalance} \times \text{currentVirtualPrice}) - \text{lastStoredTotalLpValue}$, capturing growth accrued in `CURVE_LP_STATE_ID` as well as in `CURVE_GAUGE_STATE_ID`. Harvest should then run the fee leg unconditionally against the cumulative snapshot and gate only the gauge-specific reward-claim leg on the current state.

CLIENT'S COMMENTARY

MixBytes: Marked acknowledged. The early return was removed so the fee harvesting runs in every state, but the fee base is still the gauge-staked balance only, so virtual-price growth on LP parked locally in `CURVE_LP_STATE_ID` (the reported scenario) stays uncharged — a full fix tracks total LP-equivalent (staked + local) as one cumulative snapshot. Running the fee harvest in non-gauge states adds no transfer-revert risk (the fee is bounded by `gauge.balanceOf`, hence zero and skipped when nothing is staked); the only non-gauge revert vector is the subtraction underflow under finding 2.

L-5 Initialization does not verify cross-field consistency

● LOW

● FIXED AS OF 6820a6dd

DESCRIPTION

`USDT0BridgeAdapter.initialize` accepts `_usdt0`, `_oft`, and `_lzEndpoint` as independent admin-supplied parameters with only zero-address checks. A mismatched triple (an OFT whose `token()` does not equal `_usdt0`, or an Endpoint that is not the OFT's actual Endpoint) silently misroutes funds or bricks inbound delivery.

`CurveGauge.initialize` validates only `_gauge`. The derived `lpToken`, `underlyingAsset0`, `underlyingAsset1` are trusted without check; a misconfigured gauge whose `lp_token()` returns zero, or a pool whose `coins(0)` equals `coins(1)`, silently produces a strategy with broken NAV accounting.

Each trigger is a single deployer misconfiguration at proxy initialisation, and the addresses are immutable afterwards.

RECOMMENDATION

We recommend adding non-zero and consistency checks for every derived address at initialisation, including a mainnet check that the OFT's `token()` equals the supplied USDT0 address and that the OFT's `endpoint()` equals the supplied LayerZero Endpoint address.

CLIENT'S COMMENTARY

MixBytes: Fixed on the USDT0 adapter — `initialize` takes only `_localOft` and derives `usdt0` / `lzEndpoint` from it, eliminating the mismatched-triple class. Not fixed on `CurveGauge`: `initialize` still derives `lpToken` / `underlyingAsset0` / `underlyingAsset1` from the gauge without consistency checks (non-zero `lp_token()`, `coins(0) != coins(1)`).

L-6 Dormant pool coins are excluded from `_curveGaugeNav` and `_curveLpNav`

● LOW

● ACKNOWLEDGED

DESCRIPTION

`_curveLpNav` at :78-88 and `_curveGaugeNav` at :90-102 report only the staked-LP or local-LP value, ignoring any direct holdings of `underlyingAsset0` / `underlyingAsset1` on the strategy. Under normal operation in the `CURVE_GAUGE` state the strategy holds only staked LP, but `_enterCurveLp` reads `balanceOf` of both coins and feeds them into `add_liquidity`, so any pool-coin tokens that end up on the strategy are silently folded into the next enter and accrue to incoming depositors.

Possible sources include direct donations, leftovers from earlier failed `add_liquidity` calls, residuals from partial emergency exits, and reward tokens whose address happens to equal one of the pool coins.

RECOMMENDATION

We recommend including the value of dormant pool-coin balances in the NAV view for the LP and gauge states, or alternatively sweeping idle pool coins into the pool on every harvest.

**L-7 Math.mulDiv reverts when
ICurveStableSwapNG.totalSupply() == 0**

● LOW

● FIXED AS OF 3c76ca02

DESCRIPTION

`_curveLpNav` and `_curveGaugeNav` divide by the LP token's `totalSupply` through `Math.mulDiv`, which reverts on a zero denominator. A pool that has drained to `totalSupply == 0` (a black-swan event) would cause every NAV read on the strategy to revert and block all user-facing operations until external liquidity returns.

RECOMMENDATION

We recommend short-circuiting the NAV view to return zero when the underlying pool's `totalSupply` is zero.

L-8 Uninitialised lastStored* charges fee on principal

● LOW

● FIXED AS OF 2a1d8c2f

DESCRIPTION

`_harvest` reads `lastStoredVirtualPrice` as the fee-computation baseline. When this value is zero, the numerator collapses to `currentGaugeBalance × currentVirtualPrice`, treating the entire gauge balance as gain and charging the configured fee fraction on principal.

The variable is reachable as zero through two routes: a zero-amount initial enter from `NO_ALLOCATION` (the parent's slippage check accepts `0 >= 0` on both sides) that transitions the strategy into `CURVE_GAUGE` without invoking the snapshot writes, and an emergency-management path that places the strategy into `CURVE_GAUGE` without going through `_enterCurveGauge`. If the gauge subsequently acquires balance through any path the next harvest would charge the fee on the full amount.

RECOMMENDATION

We recommend adding an early initialisation guard in harvest that seeds the snapshot from current state and returns without charging a fee when the baseline is unset.

CLIENT'S COMMENTARY

MixBytes: Fixed — `_harvest` now seeds the snapshot and returns without charging a fee when the baseline is unset (`lastStoredGaugeBalance == 0`), the recommended guard. Balance that reaches the gauge outside `_enterCurveGauge` (donation / emergency placement) is taken as principal on the next harvest rather than as gain.

L-9 Compose payload encodes instruction.amount instead of the OFT-delivered amount

● LOW

● FIXED AS OF 6820a6dd

DESCRIPTION

`quoteBridgeNativeFee` at `:94` encodes `instruction.amount` (the bridger-supplied input) into the OFT compose payload rather than the amount the OFT will actually deliver. For the audited USDT0 deployment the local and shared decimals match, the OFT's dust-removal step is a no-op, and the two values coincide. The pattern relies on this matching-decimals invariant of one particular deployment: on any future deployment where local and shared decimals differ, the OFT debits a smaller amount at the source while the compose payload still carries the original input, so the destination would credit more than was delivered.

RECOMMENDATION

We recommend reading `oftReceipt.amountReceivedLD` from `IOFT.quoteOFT` (already returned inside `quoteBridgeNativeFee`) and encoding it into the compose payload, so the credited amount tracks the OFT-delivered amount regardless of the decimals configuration.

L-10 Unnecessary safeIncreaseAllowance when a pool-coin amount is zero

● LOW

● FIXED AS OF 3c76ca02

DESCRIPTION

`_enterCurveLp` at :104-117 early-returns only when both pool-coin balances are zero. When exactly one of `amounts[0]` and `amounts[1]` is zero, the function still calls `safeIncreaseAllowance(underlyingAssetX, lpToken, 0)` on the zero-amount side. The subsequent `add_liquidity(amounts, 0)` transfers nothing for that side, so the approval has no semantic effect, but the call still incurs an external invocation of the token contract.

RECOMMENDATION

We recommend gating each `safeIncreaseAllowance` call on the corresponding `amounts[i] > 0` so the approval step matches the deposit step.

L-11 Harvest fee charges twice on the same virtual-price band after a dip and recovery

● LOW

● FIXED AS OF 2a1d8c2f

DESCRIPTION

The harvest fee charges a fee only when the virtual price has grown since the snapshot (`currentVirtualPrice > lastStoredVirtualPrice` at `_harvest :238`), and the snapshot is refreshed to the current virtual price unconditionally at the end of every harvest (`:281-282`), with no high-water mark.

When the pool's virtual price decreases and later recovers, a harvest executed while the price is below the previous snapshot lowers `lastStoredVirtualPrice` to the dipped value — the fee harvest is skipped for that call, but the tail refresh still runs. The subsequent recovery is then charged as fresh growth, re-charging the band that was already charged when the price first climbed through it.

Numeric trace (constant gauge balance `B`, fee fraction `f`), starting from snapshot virtual price 1.00:

- a harvest at the dip bottom 0.98 takes no fee (the guard `0.98 > 1.00` is false) but still refreshes the snapshot to 0.98;
- a harvest after recovery to 1.02 charges `f` on `B × (1.02 - 0.98)`, i.e. on the whole 0.98→1.02 band, whereas the net growth above the pre-dip 1.00 is only `B × (1.02 - 1.00)`. The 0.98→1.00 band — already paid for when the price first reached 1.00 — is charged a second time.

The over-charge is taken from depositors' yield in favour of the treasury. It requires both a virtual-price decrease — rare for plain (`asset_type == 0`) pools, where it only occurs on a governance `ramp_A` down-ramp, but more plausible for rate-bearing (oracle / ERC-4626) pool types — and a harvest landing while the price is below the prior snapshot. If no harvest executes during the dip, the snapshot is not lowered and no double charge occurs.

RECOMMENDATION

We recommend maintaining a high-water mark on the virtual price: do not lower `lastStoredVirtualPrice` on a harvest when the price has fallen — keep `lastStoredVirtualPrice = max(lastStoredVirtualPrice, currentVirtualPrice)` — so a recovery is charged only on growth above the previous high rather than re-charging the recovered band.

CLIENT'S COMMENTARY

MixBytes: Fixed — `lastStoredVirtualPrice` is now updated as `Math.max(currentVirtualPrice, lastStoredVirtualPrice)`, a high-water mark, so a harvest during a dip no longer lowers the baseline and a recovery is not re-charged. Minor residual: the high-water mark is applied only in `_harvest`; `_enterCurveGauge` / `_exitCurveGauge` still set the baseline to the current price, so an enter/exit landing during a dip can still re-charge the recovered band.

L-12 MorphoVault._setRewardTokens does not reject the vault-share token

● LOW

● FIXED AS OF 462f7817

DESCRIPTION

`MorphoVault._setRewardTokens` at :112-121 validates each configured reward token against `address(0)` and against `underlyingAsset`, but does not reject `morphoVault` — the strategy's own ERC-4626 vault share, which represents the entire principal position — and does not de-duplicate the list:

```
for (uint256 i = 0; i < rewardTokensLength; i++) {
    require(_rewardTokens[i] != address(0), Errors.ZeroAddress());
    require(_rewardTokens[i] != underlyingAssetCached,
RewardTokenMatchesUnderlyingAsset());
}
```

If a trusted operator (`HARVEST_MANAGER_ROLE` or the container) configures the vault-share token as a reward token, the harvest reward loop at :240-243 calls `_swapToInputTokens(morphoVault, underlyingAsset, 0, false)`, which reads the full vault-share `balanceOf` — the entire principal — and swaps it to underlying at a zero floor (see Medium #3) with no NAV gate to stop it. A single misconfiguration therefore routes the whole principal position through the router in one harvest.

The trigger is a configuration mistake by a trusted role rather than an attacker-reachable path, so likelihood is low; but the impact is loss of the principal position, and the code already rejects the symmetric `underlyingAsset` case, so the missing vault-share guard is an inconsistency worth closing. The Fluid adapter is not affected because its reward token is derived from the distributor at initialization and is not operator-settable.

RECOMMENDATION

We recommend rejecting the vault-share token in `_setRewardTokens` (`require(_rewardTokens[i] != morphoVault, ...)`), mirroring the existing `underlyingAsset` rejection, and as defense in depth rejecting `tokenIn ∈ {underlyingAsset, morphoVault}` inside `_swapToInputTokens`.

L-13 Morpho harvest charges a performance fee on recovery of prior losses

● LOW

● FIXED AS OF 462f7817

DESCRIPTION

`MorphoVault._calculateAccruedAssetsValue` at :212-224 returns the positive difference between the current converted asset value and `lastAssetsValue`, clamping a decrease to zero:

```
if (currentAssetsValue > lastAssetsValueCached) {
    return currentAssetsValue - lastAssetsValueCached;
}
return 0;
```

However, both `_harvest` (:261-263) and `manualClaim` (:318-320) then write `lastAssetsValue` unconditionally to the current value, including when the value has fallen. The checkpoint therefore ratchets down to the trough on a loss while no fee is taken, and a subsequent recovery is measured against that lowered baseline and charged as fresh yield.

Concretely, on a $100 \rightarrow 80 \rightarrow 100$ trajectory: the harvest at 80 takes no fee but moves the checkpoint to 80; the harvest back at 100 computes an accrued value of 20 and transfers the corresponding fee shares to the treasury, even though the strategy has only recovered previously-lost principal and produced no net gain above the original 100. The over-charge is taken from depositors' recovered principal in favour of the treasury. It requires a Morpho share-price decrease followed by a recovery with at least one intervening harvest, so the magnitude is bounded and the occurrence depends on the vault realizing a loss.

RECOMMENDATION

We recommend maintaining a high-water mark for the appreciation fee: do not lower `lastAssetsValue` on a harvest where the value has fallen — keep `lastAssetsValue = max(lastAssetsValue, currentAssetsValue)` — so a recovery is charged only on growth above the previous high rather than re-charging recovered principal.

L-14 Fluid harvest accrual subtraction is unclamped and can theoretically underflow-revert

● LOW

● FIXED AS OF 462f7817

DESCRIPTION

`FluidSupply._harvest` at :211-213 computes the base accrual as a bare checked subtraction:

```
vars.accruedAssetsValue =  
    IFluidToken(vars.fTokenCached).convertToAssets(  
        IERC20(vars.fTokenCached).balanceOf(address(this))  
    ) - vars.lastAssetsValue;
```

Unlike the Morpho sibling `_calculateAccruedAssetsValue`, which floors the result at zero, this subtraction underflows and reverts under Solidity 0.8 checked arithmetic whenever the fToken's converted value falls below the stored checkpoint `lastAssetsValue`. Because `harvest()` runs on every enter and exit, such a revert would also block deposits and withdrawals until the share price recovered above the checkpoint.

At the audited Fluid commit this cannot underflow: the fToken exchange price is monotonically non-decreasing — the liquidity-layer exchange price reverts rather than fall, and there is no supplier-slashing, bad-debt-socialization, or admin-reset path that would lower it. The exposure is therefore forward-looking: a future Fluid version that realizes a loss, or even a one-wei downward rounding, would revert `harvest`.

RECOMMENDATION

We recommend flooring the accrual subtraction as `accrued = current > last ? current - last : 0`, mirroring `MorphoVault._calculateAccruedAssetsValue`, so a share-price dip cannot underflow-revert the harvest.

L-15 MorphoVault.stateNav(NO_ALLOCATION) reverts where FluidSupply returns zero

● LOW

● FIXED AS OF 462f7817

DESCRIPTION

`MorphoVault.stateNav` at :123-131 handles only `UNDERLYING_ASSET_STATE_ID` and `MORPHO_VAULT_STATE_ID` and reverts `StateNotFound` for any other state id, including `NO_ALLOCATION_STATE_ID` (`bytes32(0)`):

```
if (stateId == UNDERLYING_ASSET_STATE_ID) {
    return _underlyingAssetNav();
} else if (stateId == MORPHO_VAULT_STATE_ID) {
    return _morphoVaultNav();
} else {
    revert StateNotFound(stateId);
}
```

`NO_ALLOCATION_STATE_ID` is the value `_currentStateId` holds before the first enter and after a full exit. The shared `currentStateNav()` view reads `stateNav(_currentStateId)`, so on a fully-exited or freshly-deployed Morpho strategy `currentStateNav()` reverts. The Fluid sibling handles the same sentinel explicitly and **returns 0**, so identical shared-template code is a readable `0` on one in-scope strategy and a hard revert on the other.

No in-scope flow depends on the reverting branch, but a keeper or container that reads `currentStateNav()` unconditionally — for example a heterogeneous valuation loop over several strategies — would revert on the single fully-exited Morpho member while the Fluid member returns cleanly.

RECOMMENDATION

We recommend adding an explicit `else if (stateId == NO_ALLOCATION_STATE_ID) return 0;` branch to `MorphoVault.stateNav`, mirroring `FluidSupply.stateNav`, so the lifecycle-end state is readable rather than reverting.

L-16 MorphoVault.manualClaim omits the treasury-not-set guard enforced by harvest

● LOW

● FIXED AS OF 462f7817

DESCRIPTION

The harvest dispatcher `StrategyTemplate.harvest` at :457-460 reads the treasury from the container and hard-requires it to be non-zero before taking any fee. `MorphoVault.manualClaim` at :266-321 re-implements the same fee logic inline but reads the treasury at :282 without that guard, and later transfers the fee at :314-315 :

```
if (vars.vaultTokensToTreasury > 0) {
    IERC20(vars.morphoVaultCached).safeTransfer(vars.treasury,
vars.vaultTokensToTreasury);
}
```

If the container's treasury is unset (`address(0)`) while accrued yield makes `vaultTokensToTreasury > 0` , the `safeTransfer` to the zero address reverts (`ERC20InvalidReceiver`), and because it is atomic with the Angle claim earlier in the function, the entire `manualClaim` reverts — the `MERKLE_CLAIMER_ROLE` cannot claim Angle rewards at all while the treasury is unset. The window is narrow (treasury unset during initialization or a transient rotation, coinciding with accrued yield) and recoverable as soon as the treasury is set. The sibling harvest path fails cleanly with a typed `TreasuryNotSet()` error, showing the zero-treasury state was anticipated elsewhere.

RECOMMENDATION

We recommend adding `require(vars.treasury != address(0), TreasuryNotSet())` after the treasury read in `manualClaim` , mirroring the guard in the harvest dispatcher.

L-17 Redundant fToken self-approval in `_exitFluid` accumulates an unused allowance

● LOW

● FIXED AS OF 462f7817

DESCRIPTION

`FluidSupply._exitFluid` at :189–203 increases the strategy's allowance of the fToken to spend the fToken before redeeming:

```
IERC20(fTokenCached).safeIncreaseAllowance(fTokenCached, fTokenAmountToRedeem);
IFluidToken(fTokenCached).redeem(fTokenAmountToRedeem, address(this), address(this));
```

The redeem burns the strategy's own shares (`owner == msg.sender == address(this)`), so the ERC-4626 `redeem` consumes no allowance — the strategy is not spending on behalf of a third party. The `safeIncreaseAllowance` therefore has no effect on the redeem and simply raises the strategy's fToken-to-fToken allowance on every exit, accumulating an unused, monotonically growing approval. The Morpho sibling `_exitMorphoVault` correctly omits this step.

RECOMMENDATION

We recommend removing the `safeIncreaseAllowance` call from `_exitFluid`, matching `_exitMorphoVault`.

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io

[!\[\]\(a2e36727f8541f21ccb915f9ec985b58_img.jpg\)](#) [!\[\]\(da0fdacab594686e4ee03fda11937700_img.jpg\)](#) [!\[\]\(bc71890d7ec36c1eaf7870d0ff7858d6_img.jpg\)](#) [!\[\]\(278cdbcec2e63f837a2b757fb6ccca83_img.jpg\)](#)